Welcome.

Everyone:

- Pull the updates from the course GitHub repo:
  - `cd <46120-PiWE repo>`
  - `git pull origin main`

NB:

- By attending this class, you consent to being recorded. Recording will be shared to this class and possibly other DTU students for training purposes.

46120: Scientific Programming for Wind Energy

Computational thinking and design of architecture

Ju Feng

Agenda for today

- Pull new course material ✔

- Computational thinking
- Design of architecture
- A simple example
- Begin homework

Homework in last week

- **Homework 0:** Write your own Turbine classes, do it individually.
- **Homework 1:** Write a class for your final project, do it with your group.
- **Homework optional:** Transforming your CodeCamp project with OOP.

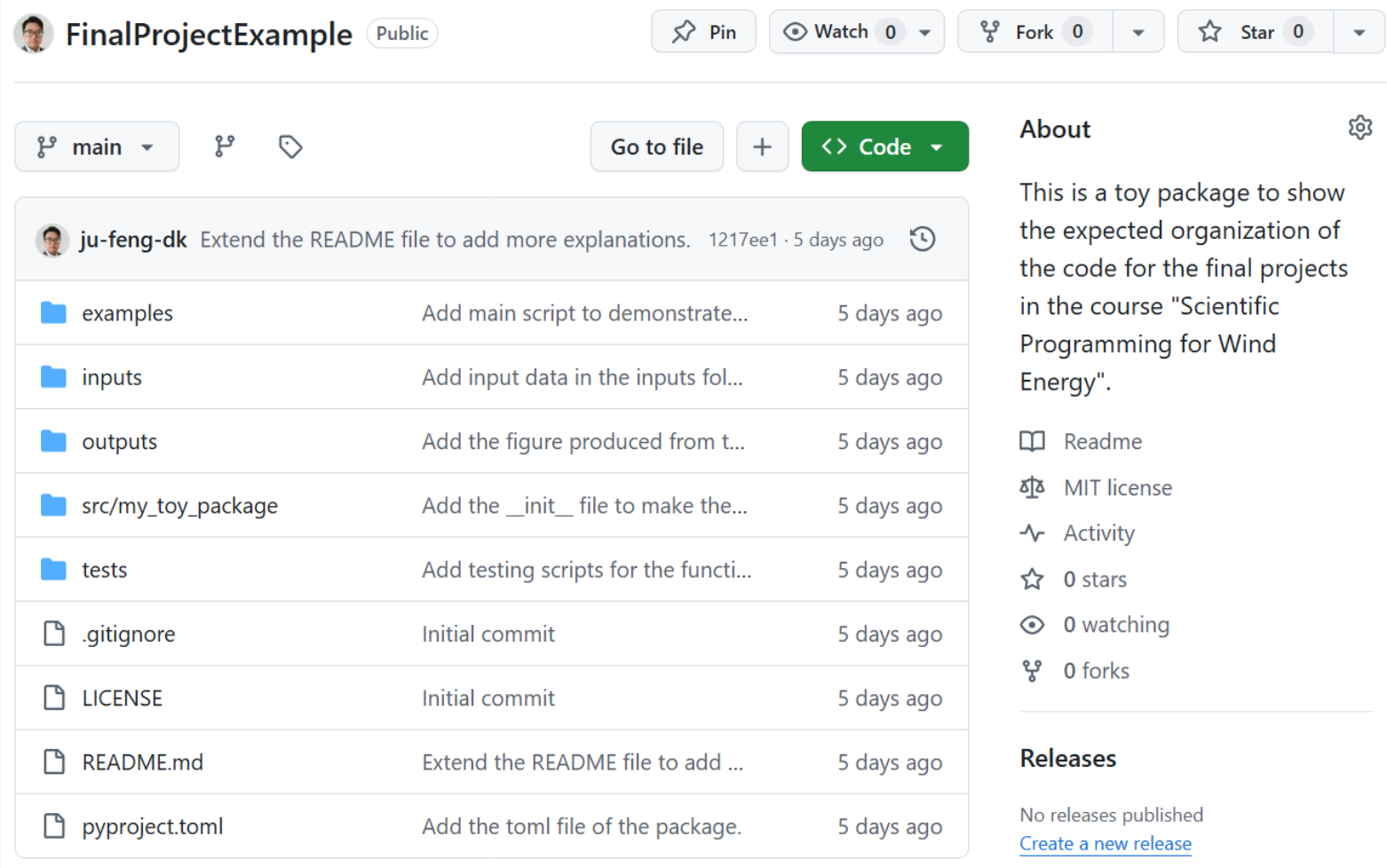
Hand in final project: by Monday, May 4th, at 23:59

A suggested development path

We have a tight schedule, think about your group's development plan. Below is a suggestion to consider:

- **Week 8 and 9**: Understand the project (background, requirements and necessary steps/models/equations), explore the provided data (load, parse, visualize), and consider the preliminary design of the architecture of your package.
- **Week 10**: Implement and refine different parts of the package (including tests).
- **Week 11**: Present your architecture and preliminary results in class, finalize the architecture and polish the code (including tests).
- **Week 12**: Refine the documentation, experiment on different computers (install the package, run the main script, check the test coverage, pylint score), fix all potential issues.

Final project example



https://github.com/ju-feng-dk/FinalProjectExample

Folder names

When you clone the final project template, you may get some folders named differently. **You should change the folder names according to the formal requirements listed in the week 8 README file**, similar to the organization of this example package:

- `inputs` folder: hosts the provided input data, that are needed for your selected final project. Data for other projects should be deleted.
- `examples` folder: hosts the `main.py` script.

Note that the core functions/classes of your package should be defined in the `src/[your_package_name]` folder, and imported in the main script to be used.

Computational thinking

Background

- What is the biggest challenge in software engineering?

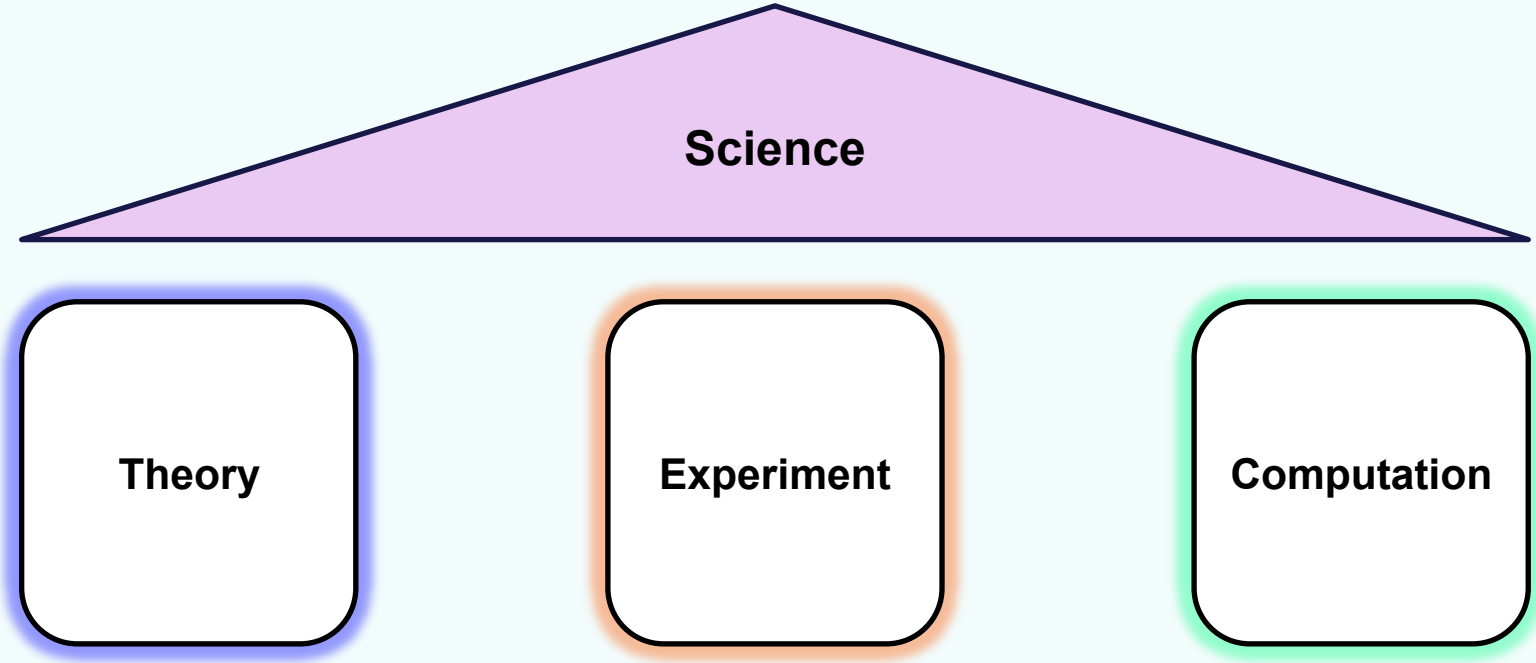
> **Managing complexity is the most important technical topic in software development.**
>
> -- Steve McConnell in *Code Complete*

- Why do we compute?

> **“We compute because we want to know and understand.”**
>
> -- Daniel Reed in *Why we compute*

> **“The purpose of computing is insight, not numbers.”**
>
> -- Richard Hamming in *Numerical Methods for Scientists and Engineers*

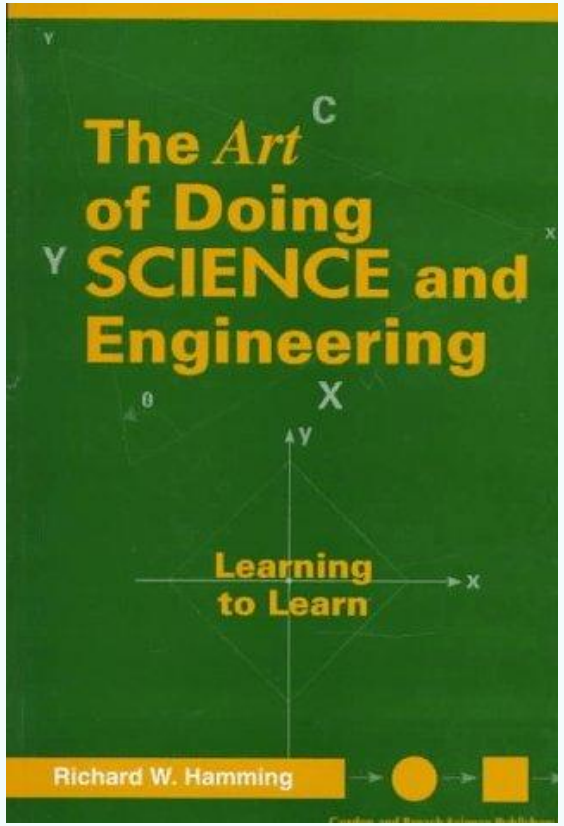
The role of computation



Together with theory and experimentation, computational science now constitutes the **“third pillar” of scientific inquiry**, enabling researchers to build and test models of complex phenomena – such as multi-century climate shifts, multidimensional flight stresses on aircraft, and stellar explosions – that cannot be replicated in the laboratory, and to manage huge volumes of data rapidly and economically.

-- From *Computational Science: Ensuring America’s Competitiveness*

Science and Engineering



Hamming, R. W. *The art of doing science and engineering: Learning to learn*. Gordon and Breach Science Publishers, 1997

"I need to discuss science vs. engineering. Put glibly:

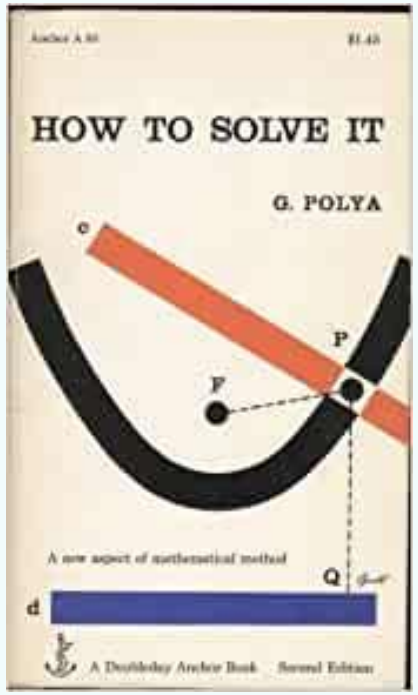
> **In science if you know what you are doing you should not be doing it. In engineering if you do not know what you are doing you should not be doing it.**

Of course, you seldom, if ever, see either pure state. All of engineering involves some creativity to cover the parts not known, and almost all of science includes some practical engineering to translate the abstractions into practice. Much of present science rests on engineering tools, and as time goes on, engineering seems to involve more and more of the science part. Many of the large scientific projects involve very serious engineering problems - the two fields are growing together!"

-- Richard Hamming in *The Art of Doing Science and Engineering*

Solving problems

- Engineering combines the fields of science and maths to **solve real world problems** that improve the world around us.



Polya, George. *How to solve it: A new aspect of mathematical method*. Princeton university press, 2004. (first in 1945)

Problem Solving Process

1. **Understand the problem (Analysis)**

2. **Devise a plan (Planning)**

3. **Carry out the plan (Implementation)**

4. **Look back (Reflection)**

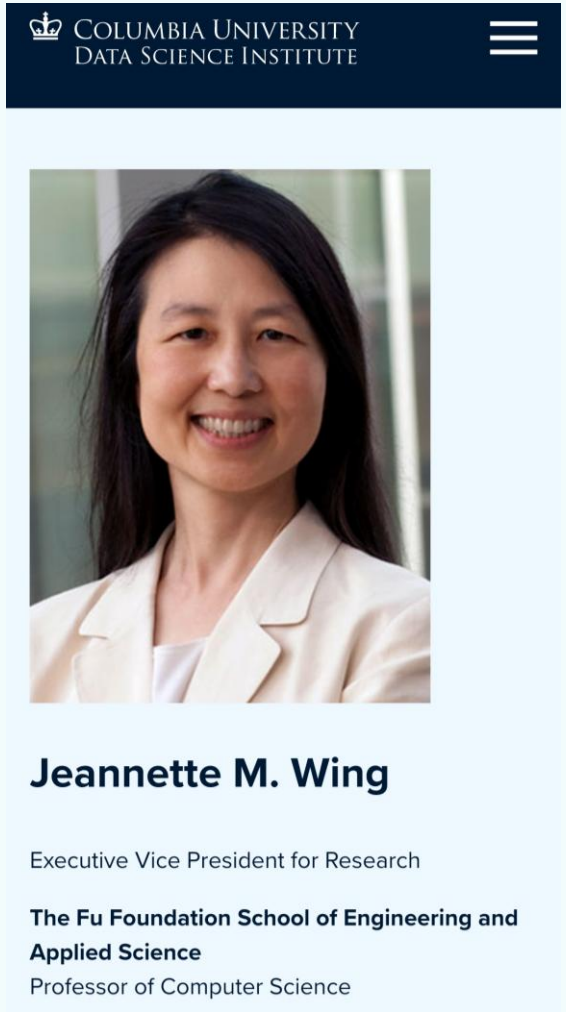
What is computational thinking?

Computational thinking is the thought processes involved in formulating a problem and expressing its solution(s) in such a way that a computer – human or machine – can effectively carry out.

-- Jeannette M. Wing

- **Thought process** (thinking)
- **Problem formulation** (what to solve)
- **Solution expression** (how to solve)
- Computer implementation (solve it)

Source: https://www.youtube.com/watch?v=V9Xy18YEK9M



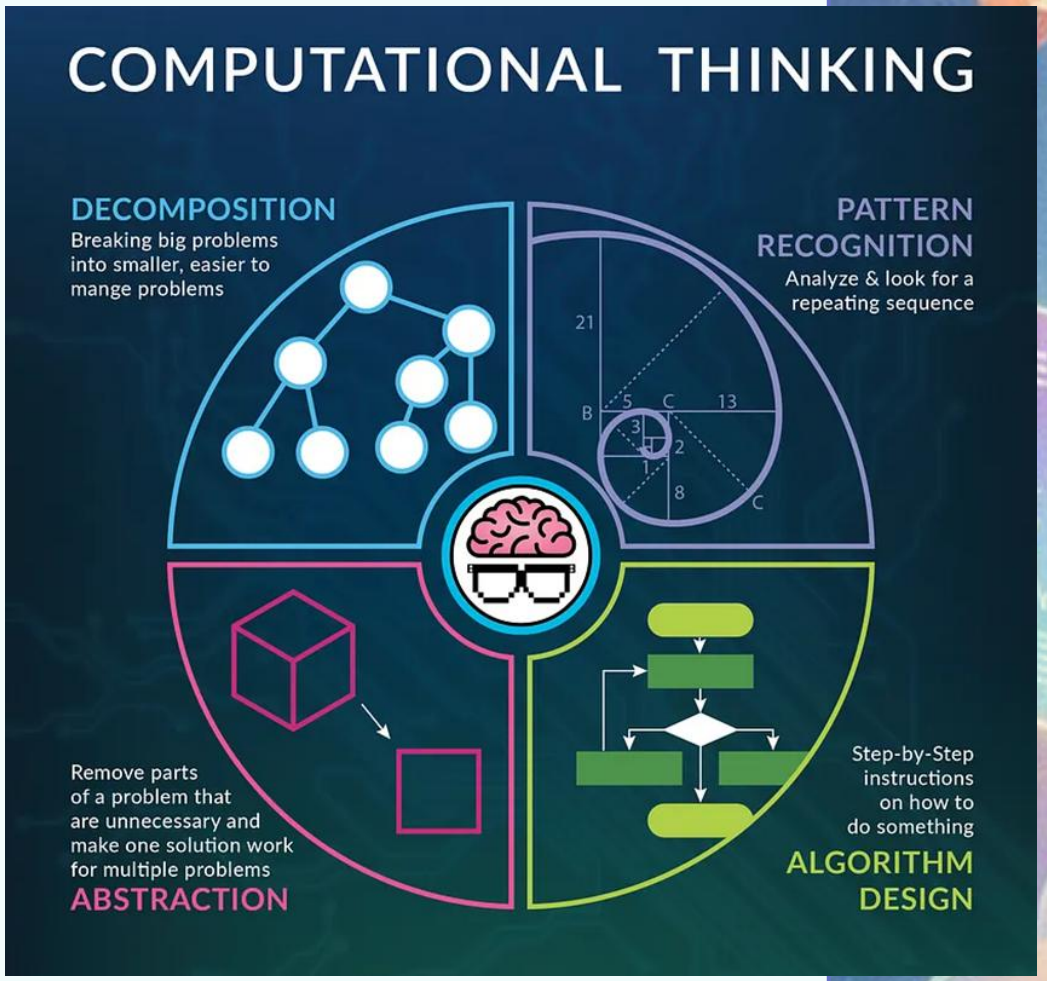
Source: https://datascience.columbia.edu/people/jeannette-m-wing/

Computational thinking

Four important concepts/techniques:

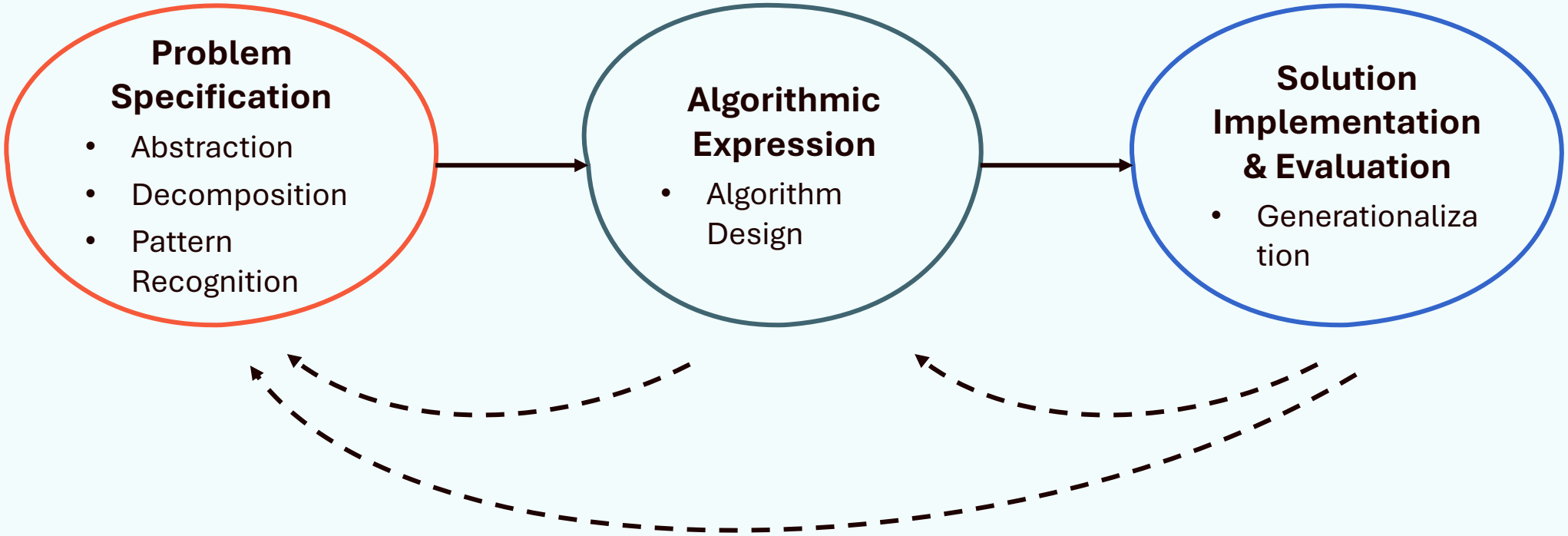
- **Abstraction**: Removing aspects of a problem that are not needed for its solution.
- **Decomposition**: Breaking a big problem down into smaller, more manageable sub-problems.
- **Pattern recognition**: Analyzing any kind of repeating elements or sequences in the problem.
- **Algorithm design**: Creating step-by-step instructions for solving the problem.
- **Generalization**: Extending a solution for a particular problem to other kinds of problems.

Source: Sethi, Ricky J. *Essential Computational Thinking: Computer Science from Scratch*. Cognella Academic Publishing, 2020.



Source: https://medium.com/@sarahyam/how-can-we-use-computational-thinking-to-solve-business-problems-6e8e37913804hat

Computational thinking steps



Adapted from: Sethi, Ricky J. *Essential Computational Thinking: Computer Science from Scratch*. Cognella Academic Publishing, 2020.

Computational thinking approach

1. Problem Specification

- **Computational Thinking Principles:** Problem Analysis and Abstraction
- **Computer Science Ideas:** Model Development using Decomposition and Pattern Recognition
- **Software Engineering Techniques**: Problem Requirements, Problem Specifications, UML diagrams, etc.

2. Algorithmic Expression

- **Computational Thinking Principles:** Data representation + Algorithm = Computational solution
- **Computer Science Ideas:** Systematically process information using modularity, flow control (including sequential, selection, and iteration), recursion, encapsulation, and parallel computing
- **Software Engineering Techniques:** Flowcharts, Pseudocode, Data Flow Diagrams, Class-responsibility-collaboration (CRC) cards for Class Diagrams, etc.

3. Solution Implementation & Evaluation

- **Computational Thinking Principles:** Systematic Testing and Generalization
- **Computer Science Ideas:** efficiency analysis, debugging, testing, etc.
- **Software Engineering Techniques**: Code Reviews, Refactoring, Test Suites, Quality Assurance (QA), etc.

From: Sethi, Ricky J. *Essential Computational Thinking: Computer Science from Scratch*. Cognella Academic Publishing, 2020.

Develop your computational thinking skills (through practicing)

Practice	What to Do	Wind Energy Example
Decomposition	Break big problems into smaller ones	Split "model wind farm" into: load data → model wind → model wakes → compute powers
Pattern Recognition	Look for similarities and repetitions	Notice all sensor data follows same pattern: read → clean → analyze
Abstraction	Focus on essentials, ignore details	For power calculation: use wind speed & power curve, not molecular physics
Algorithm Design	Plan step-by-step solutions first	Sketch data flow for AEP calculation before coding, design the system before coding

Mindset Shift:
From *"I need to write code"* →
"I need to solve this problem systematically"

Design of architecture

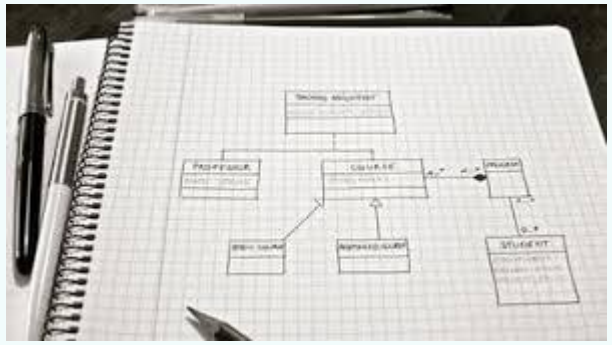
What is software architecture?

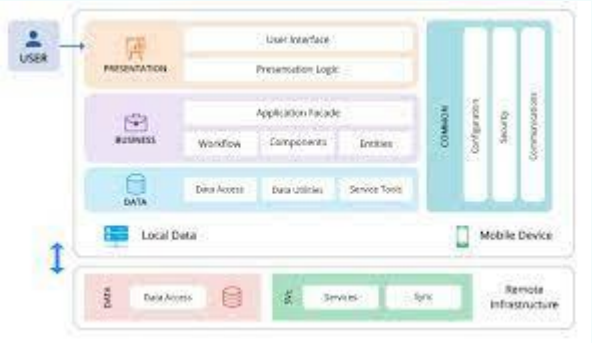
All architecture is design, but not all design is architecture. Architecture represents the set of **significant design decisions** that shape the form and the function of a system, where significant is measured by cost of change.

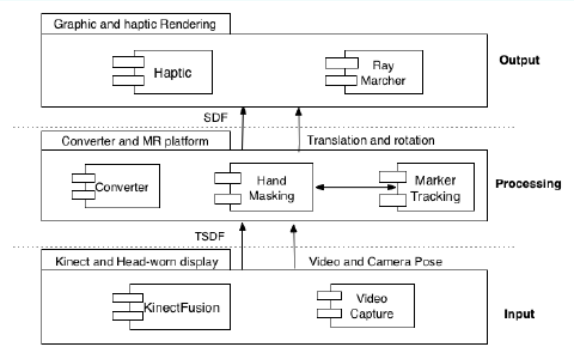
-- Grady Booch in https://handbookofsoftwarearchitecture.com

The software architecture of a system is the **set of structures** needed to reason about the system, which comprise software elements, relations among them, and properties of both.

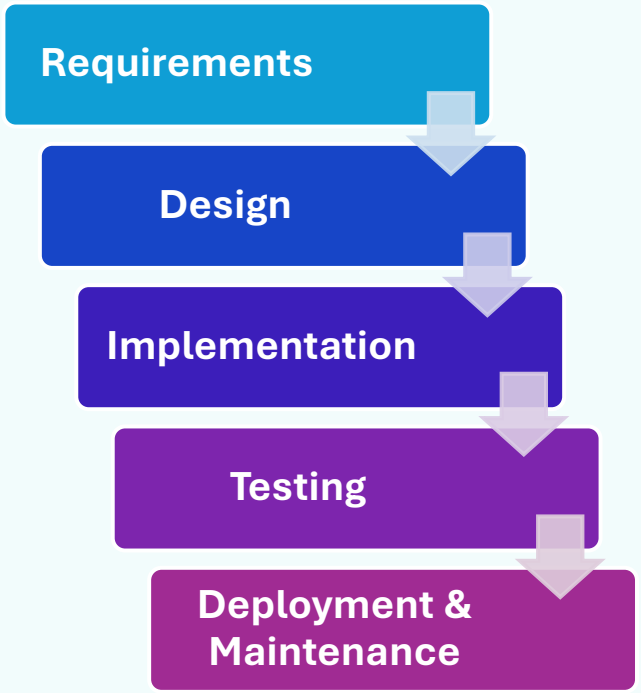
-- Len Bass, et al. in *Software architecture in practice*

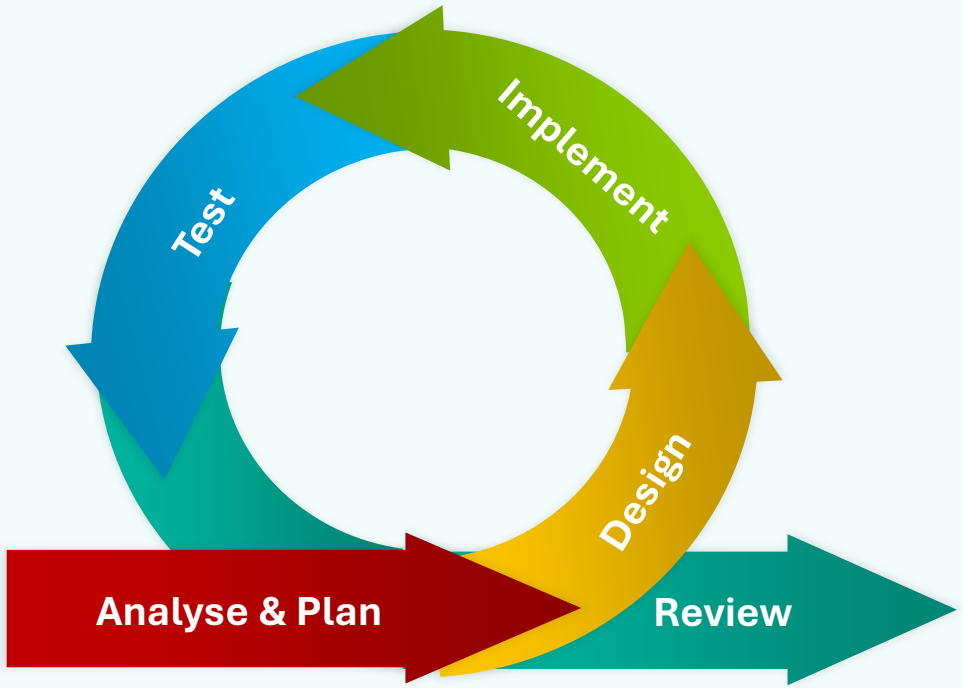






From Waterfall to Agile





Waterfall methodology
Sequential, linear approach where each phase must be fully completed before the next begins.

Agile methodology
Iterative, flexible approach with short development cycles and frequent feedback.

Do we need software architecture?

- From waterfall to Agile, do we still need software architecture? Definitely, yes.

> In preparing for battle, I have always found that plans are useless but planning is indispensable.
>
> -- Dwight D. Eisenhower

> Big up-front design is dumb! No up-front design is dumber!
>
> -- Dave Thomas

Why is software architecure important?

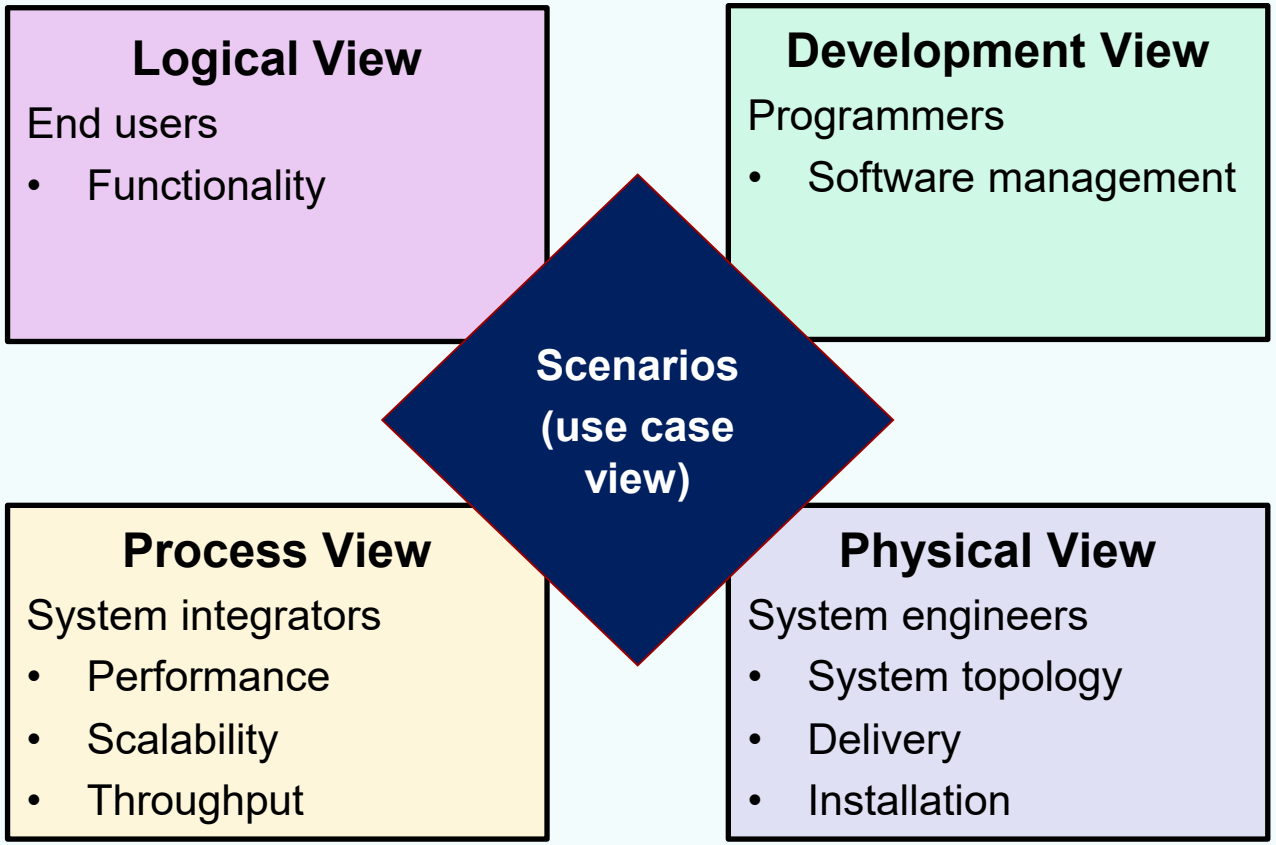
1. Allow reasoning about and managing changes.
2. Enhance communications among stakeholders.
3. Is a carrier of the earliest and hence most fundamental, hardest-to-change design decisions.
4. Provide the basis for evolutionary prototyping.
5. Focus attention on the assembly of components, rather than simply on their creation.
6. Channel the creativity of developers, reducing design and system complexity.

In general, a good software architecture can help increasing a software product's quality in terms of: Modularity, Understandability, Reusability, Extensibility, Modifiability, Testability, etc.

Selected from: Bass, L., Clements, P., & Kazman, R. (2003). *Software architecture in practice*. Addison-Wesley Professional.

The 4 + 1 view model

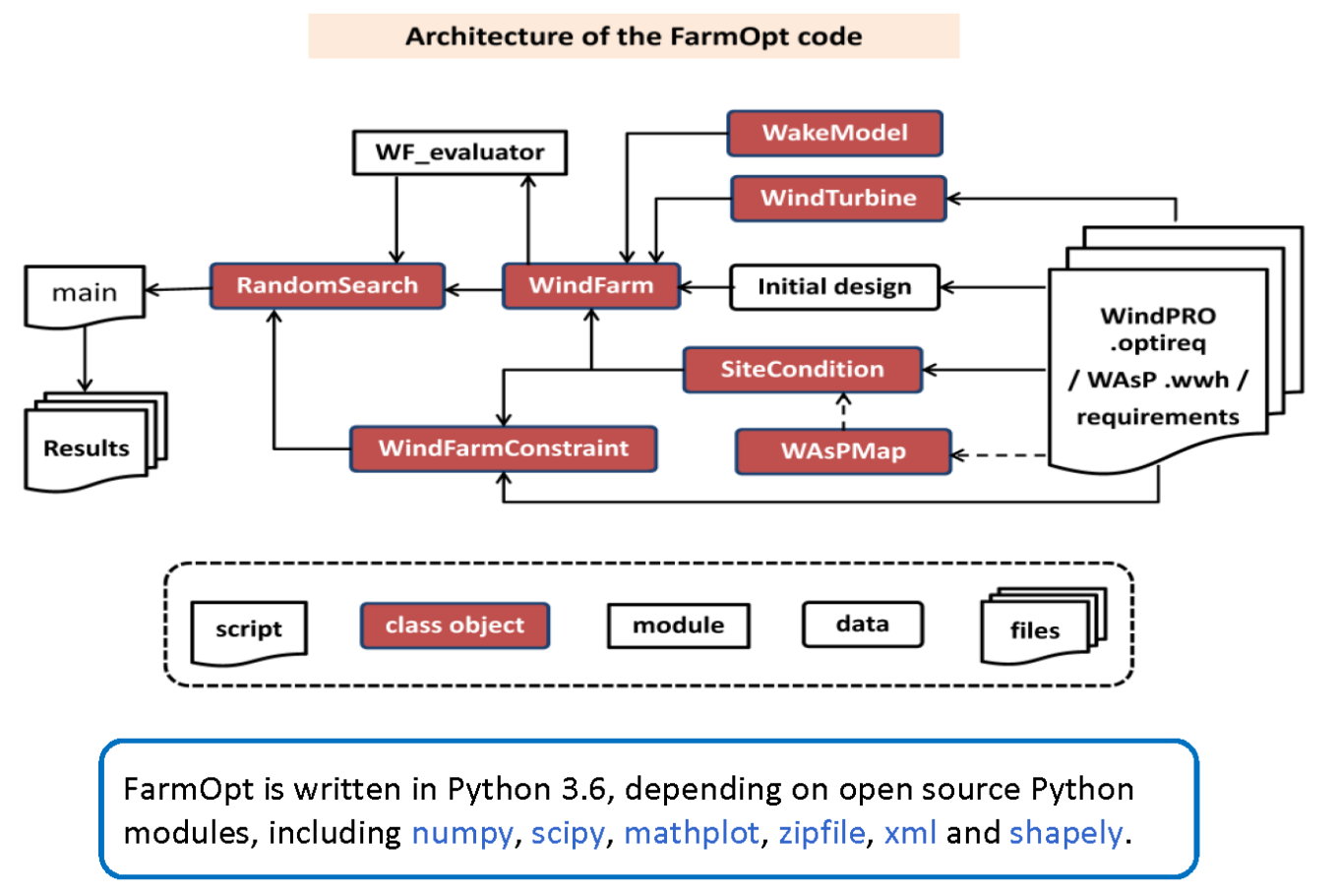
- Describe a software architecture using five concurrent views, each addresses a specific set of concerns of interest to different stakeholders in the system.



Adapted from: Kruchten, Philippe B. "The 4+1 view model of architecture." *IEEE software*. 12.6 (1995): 42-50.

Architecture diagram

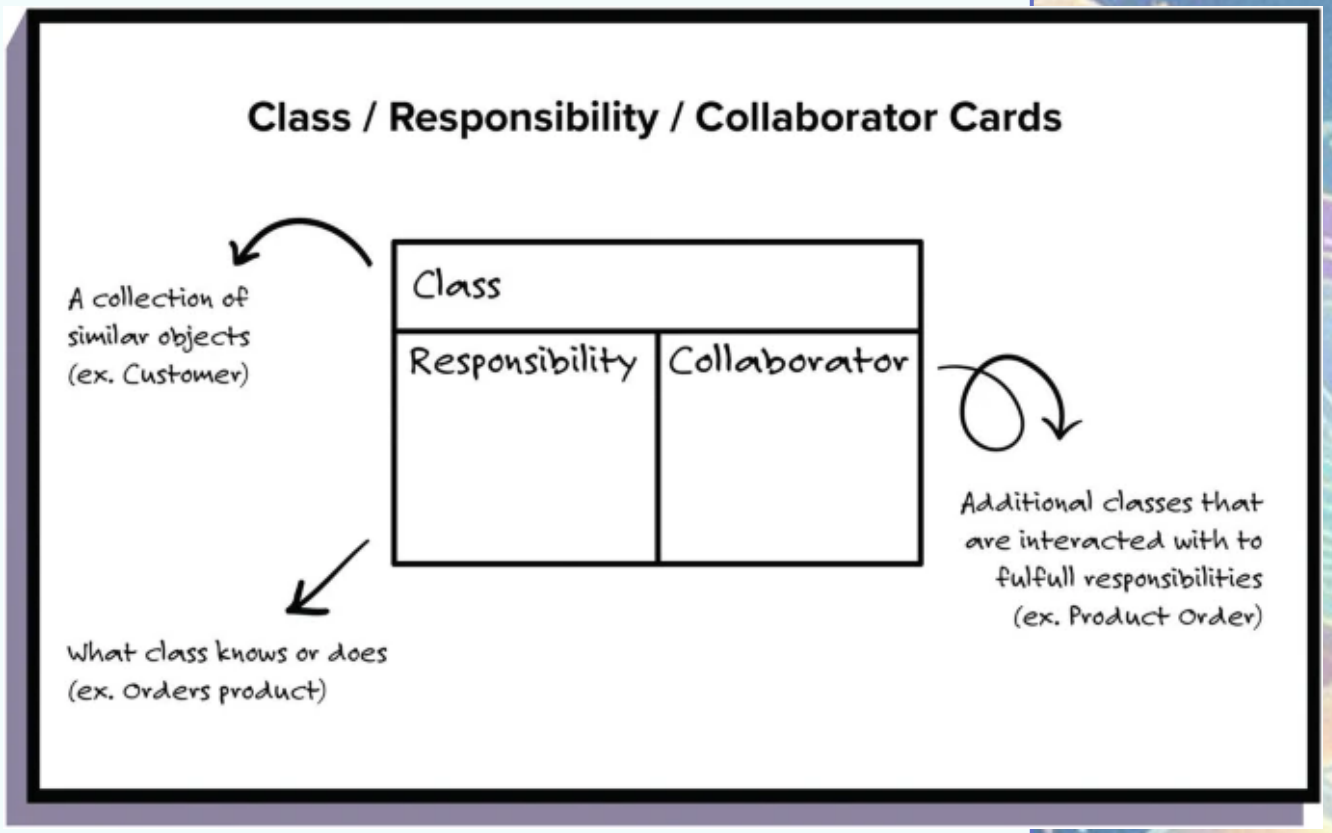
- Architecture can be documented using a diagram with boxes and arrows.
- It can be drawn on paper or whiteboard, using PowerPoint or Zoom whiteboard, or Miro, or draw.io (draw.io), or mermaid (mermaid.live).
- Unified Modeling Language (UML) or other models/tools may not be needed for scientific computing.



A personal example for the FarmOpt code developed in 2016.

Class, Responsibilities, Collaborators (CRC) Cards

- First, two or more team members write down the names of the most critical classes involved in the feature on index cards.
- Second, the cards are fleshed out with lists of the responsibilities of each class and the names of collaborators (i.e., other dependent classes).
- Third, team members perform a role-playing exercise and assume the role of one or more classes while playing out a plausible scenario of the design.



Source: Dalton, J. (2018). *Great Big Agile: An OS for Agile Leaders*. Apress.

Design patterns

- In software engineering, a software design pattern or design pattern is a general, reusable solution to a commonly occurring problem in many contexts in software design.
- Design patterns can be viewed as formalized best practices that the programmer may use to solve common problems when designing a software application or system.
- Design patterns gained popularity in computer science after the book *Design Patterns: Elements of Reusable Object-Oriented Software* was published in 1994 by the so-called "Gang of Four" (Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides), which is frequently abbreviated as "GoF".

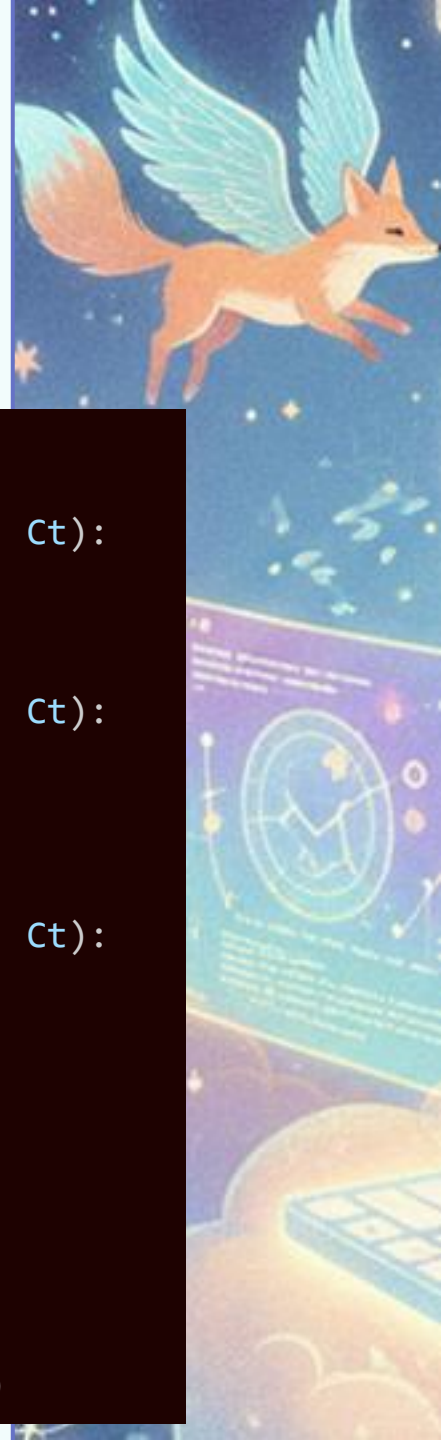
Source: Software design pattern - Wikipedia

Source: https://en.wikipedia.org/wiki/Design_Patterns

Strategy Pattern: Interchangeable Algorithms/Methods

- Define a family of algorithms/ methods, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- **Key Idea:** Define interchangeable algorithms that can be selected at runtime.
- **Benefits:**
  - Easy comparison of different scientific models
  - Research flexibility without code changes
  - Clean separation between algorithm and application

```python
class WakeModel(ABC):
    @abstractmethod
    def calculate_deficit(self, upstream, downstream, Ct):
        pass

class JensenWake(WakeModel):
    def calculate_deficit(self, upstream, downstream, Ct):
        # Jensen 1983 model implementation
        return velocity_deficit

class LarsenWake(WakeModel):
    def calculate_deficit(self, upstream, downstream, Ct):
        # Larsen 1988 model implementation
        return velocity_deficit

class WindFarmSimulation:
    def __init__(self, wake_model: WakeModel):
        self.wake_model = wake_model

    def run_simulation(self):
        # Use the selected wake model
        return self.wake_model.calculate_deficit(...)
```

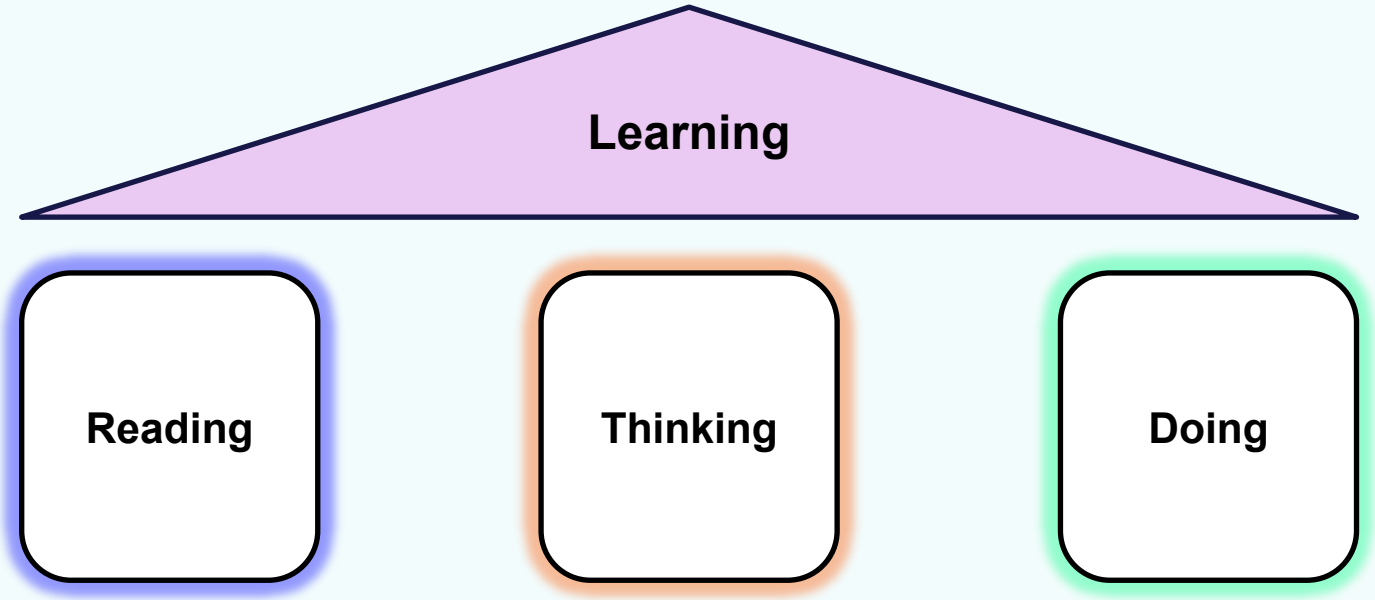


Adapter Pattern: Bridging Incompatible Interfaces

- Adapter (also called as wrapper, translator) pattern: Convert the interface of a class into another interface clients expect.
- **Key Idea:** Make incompatible interfaces work together without changing existing code.
- **Benefits:**
  - Integrates legacy scientific codes (Fortran, C)
  - Wraps external libraries and APIs
  - Enables gradual modernization of codebase
  - Provides consistent interfaces across different implementations

```python
# Legacy library (incompatible)
class OldTurbineLib:
    def compute(self, ws, dia):
        return ws * dia * 0.5

# Modern interface we want
class PowerCalculator:
    def calculate_power(self, wind_speed, rotor_diameter):
        pass

# Adapter bridges the gap
class LegacyAdapter(PowerCalculator):
    def __init__(self):
        self.old_lib = OldTurbineLib()

    def calculate_power(self, wind_speed, rotor_diameter):
        return self.old_lib.compute(wind_speed, rotor_diameter)

# Usage - same modern interface
calc = LegacyAdapter()
power = calc.calculate_power(12.5, 120)
```

How to develop computational thinking?



学而不思则罔，
思而不学则殆。

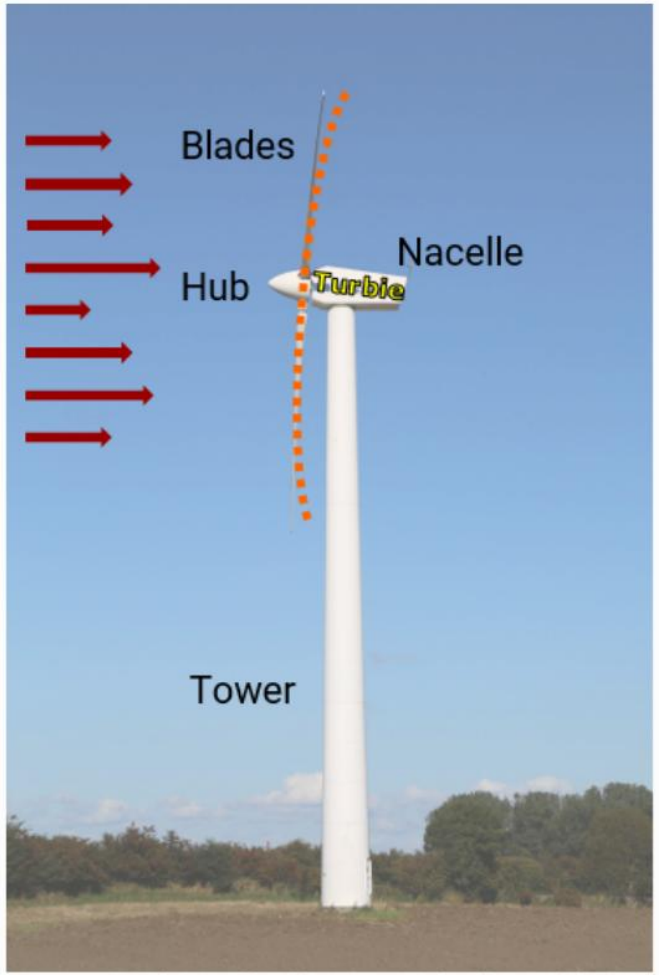
– 孔子

Learning without thought is labor lost;
thought without learning is perilous.

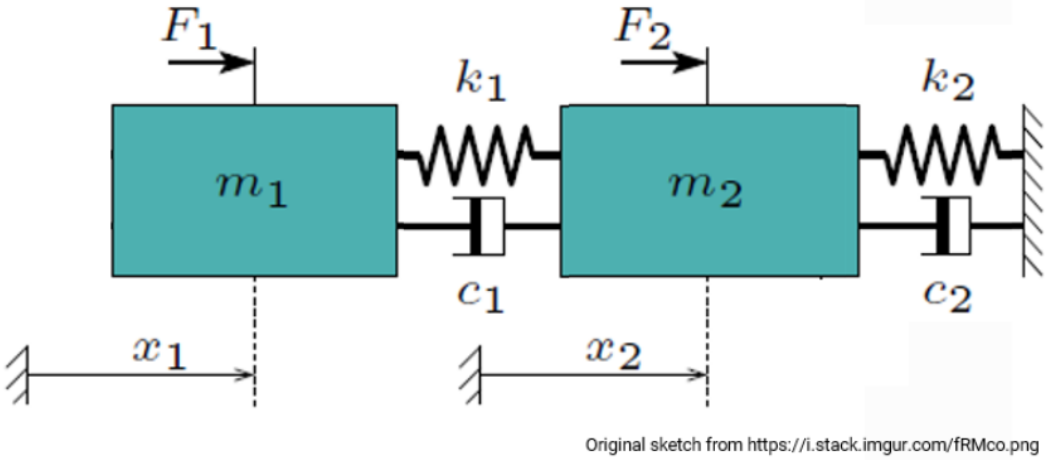
- Confucius

A Simple Example

The Codecamp project



Original picture from https://en.wind-turbine-models.com/

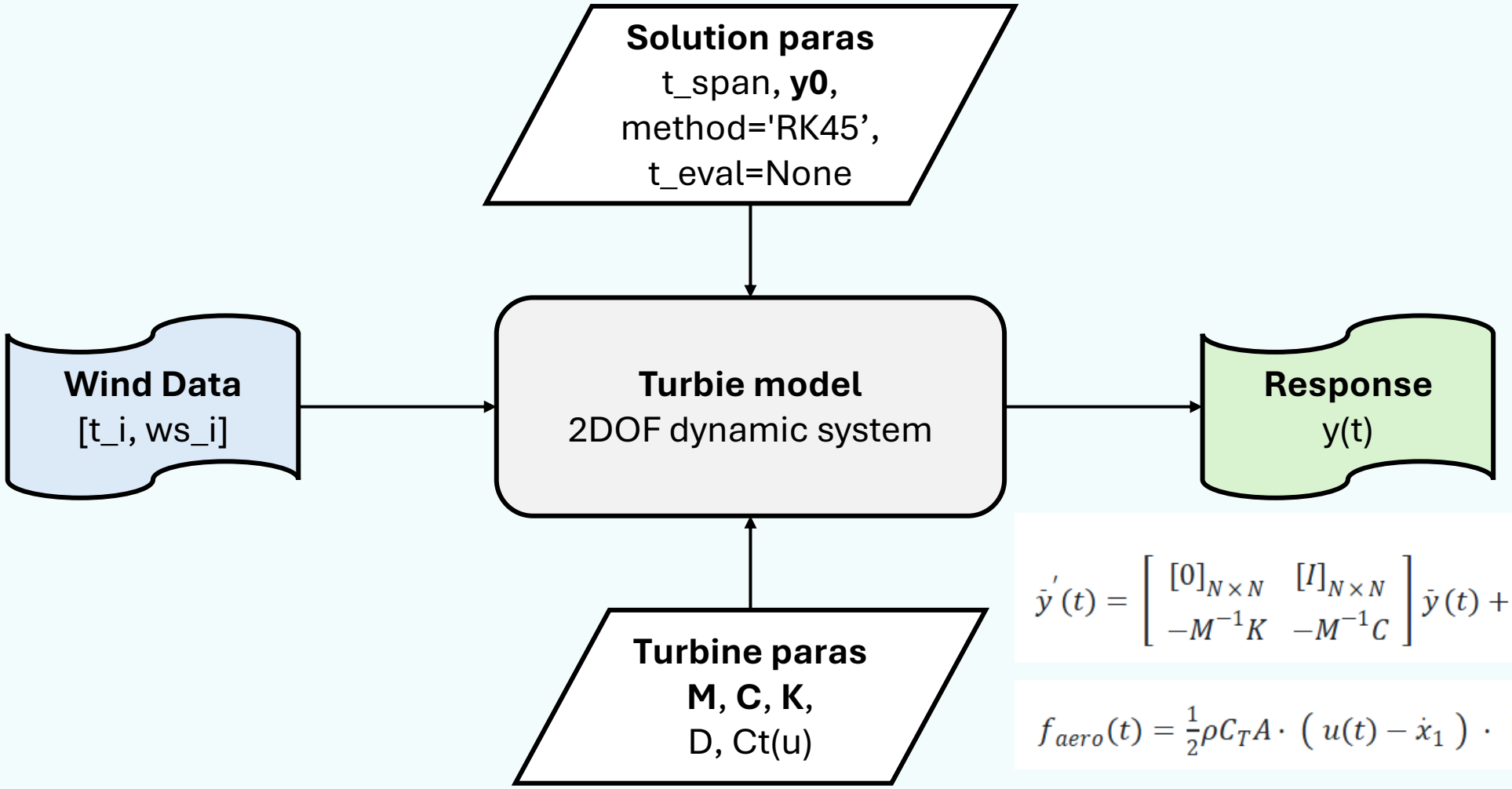


Original sketch from https://i.stack.imgur.com/fRMco.png

$$\bar{y}'(t) = \begin{bmatrix} [0]_{N\times N} & [I]_{N\times N} \\ -M^{-1}K & -M^{-1}C \end{bmatrix} \bar{y}(t) + \begin{bmatrix} 0 \\ M^{-1}\bar{F}(t) \end{bmatrix},$$

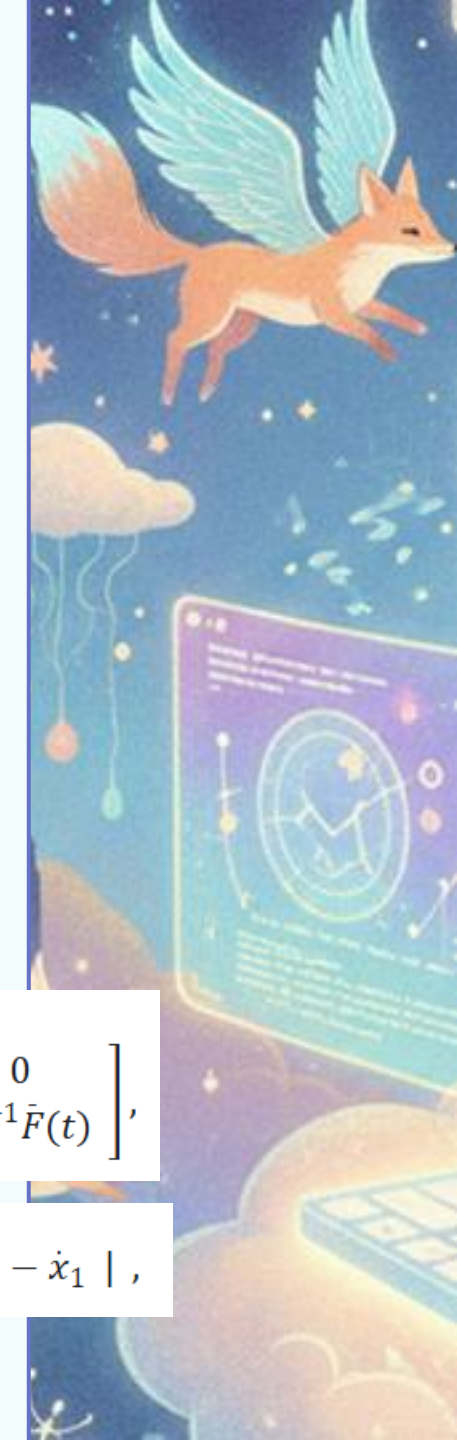
Your task for this project is to build a Turbie module within Python housing the equations required to simulate the system. Then, you can import the module and its functions into another Python script. You will then apply the provided wind files to the model to determine how the blade and tower deflections vary with the wind speed and turbulence intensity (TI).

Think about the system

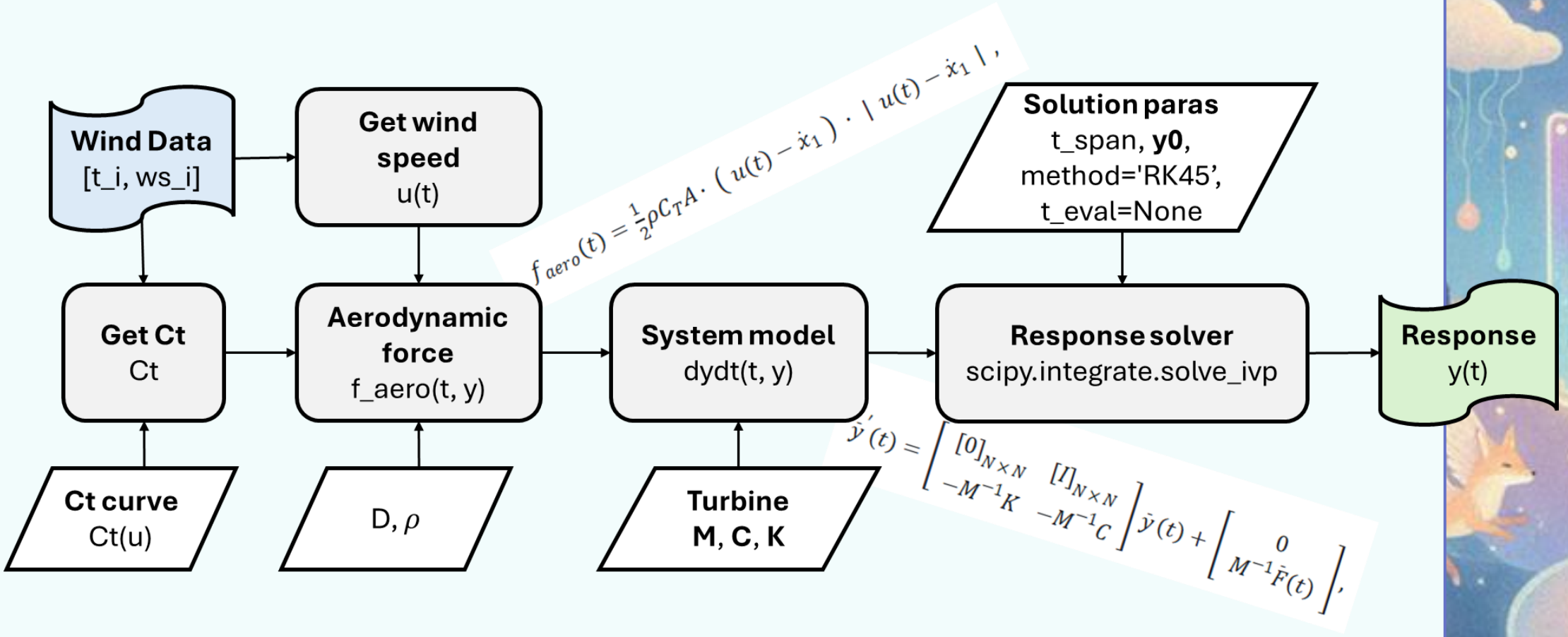


$$\dot{\bar{y}}(t) = \begin{bmatrix} [0]_{N\times N} & [I]_{N\times N} \\ -M^{-1}K & -M^{-1}C \end{bmatrix} \bar{y}(t) + \begin{bmatrix} 0 \\ M^{-1}\bar{F}(t) \end{bmatrix},$$

$$f_{aero}(t) = \frac{1}{2}\rho C_T A \cdot \left(u(t) - \dot{x}_1 \right) \cdot \left| u(t) - \dot{x}_1 \right|,$$

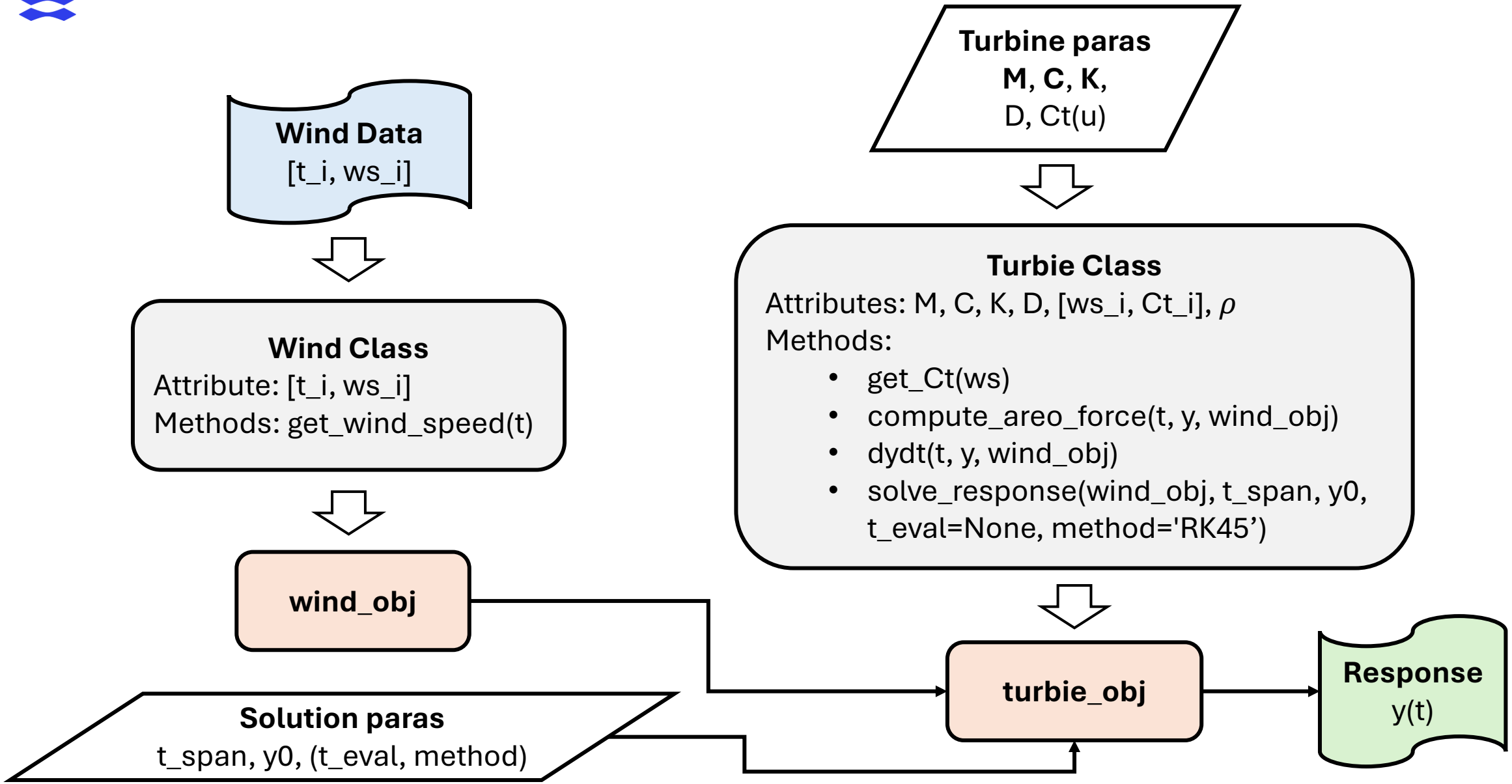


Module design

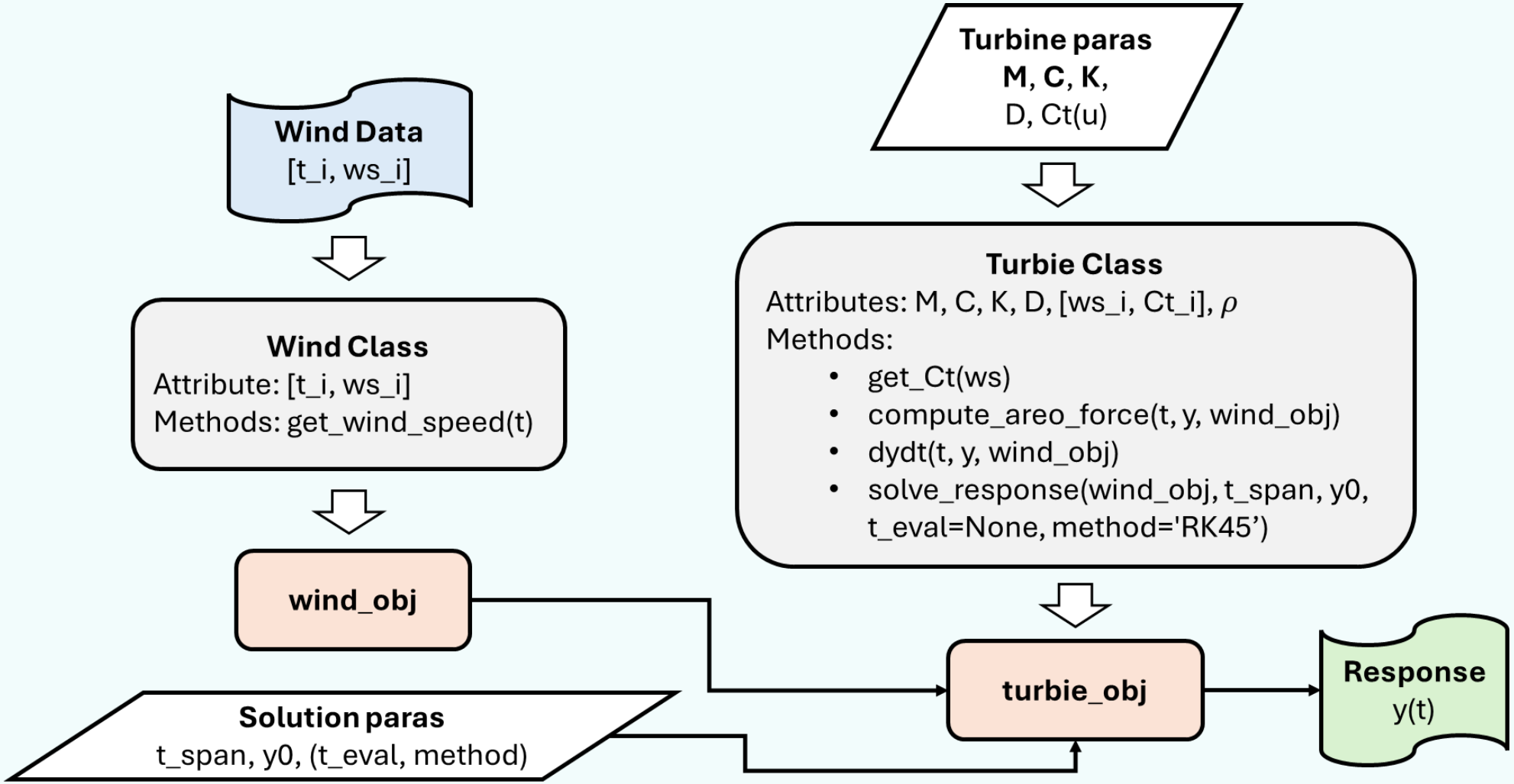


```python
# Define fixed paramters
D_rotor, rho
M, C, K

def load_ct_curve(file_path):
    # Load the Ct_curve data
    pass

def load_wind_speed_data(file_path):
    # Load the wind speed data
    pass

def get_ct(wind_speed):
    # Interpolate Ct from Ct_curve based on wind speed
    pass

def get_wind_speed(t):
    # Interpolate wind speed from ws_array based on time t
    # Assume ws_array is loaded and available globally
    pass

def compute_aero_force(t, y):
    # Compute aerodynamic forces based on current state y and time t
    pass

def dydt(t, y):
    # Define the system of ODEs, use compute_areo_force
    pass

def solve_response(dydt, y0, t0, tf, t_eval=None, method='RK45'):
    # Solve ODE using scipy's solve_ivp
    pass
```

The Turbie module


Wind Data
[t_i, ws_i]
Turbine paras
M, C, K,
D, Ct(u)
Wind Class
Attribute: [t_i, ws_i]
Methods: get_wind_speed(t)
Turbie Class
Attributes: M, C, K, D, [ws_i, Ct_i], ρ
Methods:
• get_Ct(ws)
• compute_areo_force(t, y, wind_obj)
• dydt(t, y, wind_obj)
• solve_response(wind_obj, t_span, y0, t_eval=None, method='RK45')
wind_obj
turbie_obj
Response
y(t)
Solution paras
t_span, y0, (t_eval, method)

OOP solution



Questions?

Homework for this week.

Homework.

- **Homework 0**: Design the architecture of your final project and document it in a diagram. Note that this will be up to changes when you progress more with your final project, don’t treat it as final and fixed.
- **Homework 1**: Keep working on your final projects!

Any questions?

References

- McConnell, Steve. *Code complete*. Pearson Education, 2004.
- Daniel Reed, (2011) *Why We Compute* [link: https://cacm.acm.org/blogcacm/why-we-compute/]
- Hamming, R. W. *Numerical Methods For Scientists And Engineers*, 2nd Edition. Dover Publications, 1973
- United States. President's Information Technology Advisory Committee. *Computational Science: Ensuring America's Competitiveness*. National Coordination Office for Information Technology Research & Development, 2005.
- Hamming, R. W. The *art of doing science and engineering: Learning to learn*. Gordon and Breach Science Publishers, 1997.
- Polya, George. *How to solve it: A new aspect of mathematical method*. Princeton university press, 2004.
- Sethi, Ricky J. *Essential Computational Thinking: Computer Science from Scratch*. Cognella Academic Publishing, 2020.
- Grady Booch. *Handbook of Software Architecture*. [link: https://handbookofsoftwarearchitecture.com/]
- Bass, L., Clements, P., & Kazman, R. (2003). *Software architecture in practice*. Addison-Wesley Professional.
- Kruchten, Philippe B. "The 4+1 view model of architecture." IEEE software. 12.6 (1995): 42-50.
- Dalton, J. (2018). *Great Big Agile: An OS for Agile Leaders*. Apress.
- Confucius. *The Analects*, ca. 500 B.C.E [Available at: https://classics.mit.edu/Confucius/analects.html]